

ER DIAGRAM & RELATIONAL SCHEMA DESIGN

AIRLINES MANAGEMENT SYSTEM

Modelling the airline reservation and operations application as a complete Entity-Relationship diagram and converting it into a normalised relational schema.

Submitted By	Pavan Reddy T N
Roll Number	2511021240037
Course	BCA AI/ML
Subject	Database Management System
Assignment	Assignment 2 — ER Diagram & Relational Schema

01 TASK 1

Entities, Attributes & Primary Keys

The core entities of the Airlines Management System, with their attributes and primary keys identified.

◆ PASSENGER

Attribute	Data Type	Key	Description
passenger_id	INT	PK	Unique identifier for each passenger
name	VARCHAR(100)	—	Full name of the passenger
email	VARCHAR(120)	—	Contact email, used for login
phone	VARCHAR(15)	—	Contact phone number

date_of_birth	DATE	—	Passenger's date of birth
nationality	VARCHAR(50)	—	Nationality of the passenger

◆ AIRPORT

Attribute	Data Type	Key	Description
airport_id	INT	PK	Unique identifier for each airport
airport_code	CHAR(3)	—	IATA code, e.g. BLR (unique)
airport_name	VARCHAR(120)	—	Full name of the airport
city	VARCHAR(80)	—	City where the airport is located
country	VARCHAR(60)	—	Country where the airport is located

◆ AIRCRAFT

Attribute	Data Type	Key	Description
aircraft_id	INT	PK	Unique identifier for each aircraft
model	VARCHAR(60)	—	Aircraft model, e.g. A320
manufacturer	VARCHAR(60)	—	Manufacturer, e.g. Airbus
seat_capacity	INT	—	Total number of passenger seats

◆ FLIGHT

Attribute	Data Type	Key	Description
flight_id	INT	PK	Unique identifier for each scheduled flight
flight_no	VARCHAR(10)	—	Public flight number, e.g. AI505
departure_time	DATETIME	—	Scheduled departure date and time
arrival_time	DATETIME	—	Scheduled arrival date and time
status	VARCHAR(30)	—	Scheduled / Delayed / Departed / Landed / Cancelled
base_fare	DECIMAL(10,2)	—	Standard fare for the flight

◆ BOOKING

Attribute	Data Type	Key	Description
booking_id	INT	PK	Unique identifier for each booking
booking_date	DATETIME	—	Date and time the booking was made
status	VARCHAR(30)	—	Confirmed / Cancelled / Completed
total_amount	DECIMAL(10,2)	—	Total value of the booking

◆ PAYMENT

Attribute	Data Type	Key	Description
payment_id	INT	PK	Unique identifier for each payment

payment_method	VARCHAR(30)	—	UPI / Card / Net Banking
amount	DECIMAL(10,2)	—	Amount paid
payment_status	VARCHAR(30)	—	Pending / Paid / Failed / Refunded

◆ BAGGAGE

Attribute	Data Type	Key	Description
baggage_id	INT	PK	Unique identifier for each baggage item
tag_no	VARCHAR(20)	—	Unique baggage tag number
weight_kg	DECIMAL(5,2)	—	Weight of the baggage in kilograms
baggage_status	VARCHAR(30)	—	Checked-in / Loaded / Arrived / Collected

Note: linking columns such as *passenger_id* in *Booking* or *booking_id* in *Payment* are deliberately not listed as entity attributes here. At the ER level they are expressed as relationships (Section 2) and only appear as foreign keys once the schema is derived in Section 4.

02 TASK 2

Relationships & Cardinalities

Seven relationships connect the seven entities above. Six are one-to-many; one — *Booking and Flight* — is many-to-many and is resolved into its own table in Section 4.

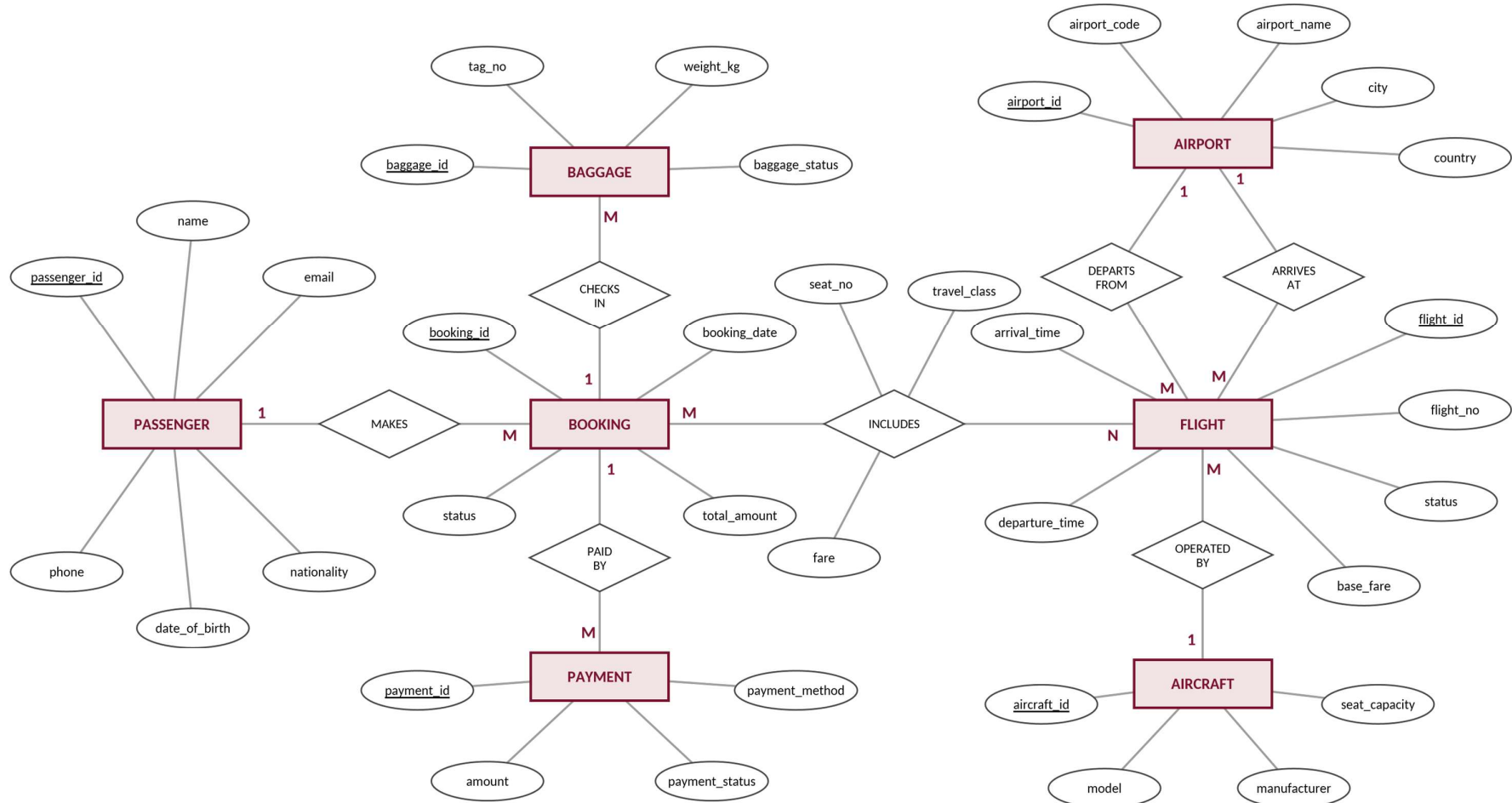
Relationship	Participating Entities	Cardinality	Meaning
MAKES	Passenger → Booking	1 : M	A passenger may make many bookings; each booking is made by exactly one passenger.
INCLUDES	Booking ↔ Flight	M : N	A booking can cover many flights (e.g. a return trip or connection), and a flight appears in many bookings. Carries its own attributes (seat_no, travel_class, fare).
PAID BY	Booking → Payment	1 : M	A booking can have more than one payment record, e.g. retries or refunds; each payment belongs to one booking.
CHECKS IN	Booking → Baggage	1 : M	A booking can have several baggage items checked in; each baggage item belongs to exactly one booking.
OPERATED BY	Aircraft → Flight	1 : M	An aircraft operates many flights over time; each flight is operated by exactly one aircraft.
DEPARTS FROM	Airport → Flight	1 : M	An airport is the origin of many flights; each flight departs from exactly one airport.
ARRIVES AT	Airport → Flight	1 : M	An airport is the destination of many flights; each flight arrives at exactly one airport.

Note: no one-to-one (1:1) relationship occurs naturally in this domain — every association either links one record to many related records, or (in the case of *INCLUDES*) links many records on each side. Airport takes part in Flight twice, once as origin and once as destination.

03 TASK 3

Complete ER Diagram

Drawn in Chen notation — entities as rectangles, relationships as diamonds, attributes as ovals, and primary keys underlined.



Relational Schema

The ER diagram is converted into tables using three standard mapping rules.

- **Entities become tables:** each entity turns into a table, and its attributes become columns; the entity's primary key stays the table's primary key.
- **1:M relationships become foreign keys:** the primary key of the "one" side is placed as a foreign key on the table of the "many" side (e.g. `passenger_id` on `Bookings`, `booking_id` on `Payments` and `Baggage`, and `aircraft_id` plus two airport keys on `Flights`).
- **The M:N relationship becomes a new table:** `INCLUDES` is converted into an associative table, `Tickets`, holding the primary keys of both `Bookings` and `Flights` as foreign keys, plus the relationship's own attributes (`seat_no`, `travel_class`, `fare`) and its own surrogate primary key, `ticket_id`.

Figure 2: Relational Schema — Airlines Management System

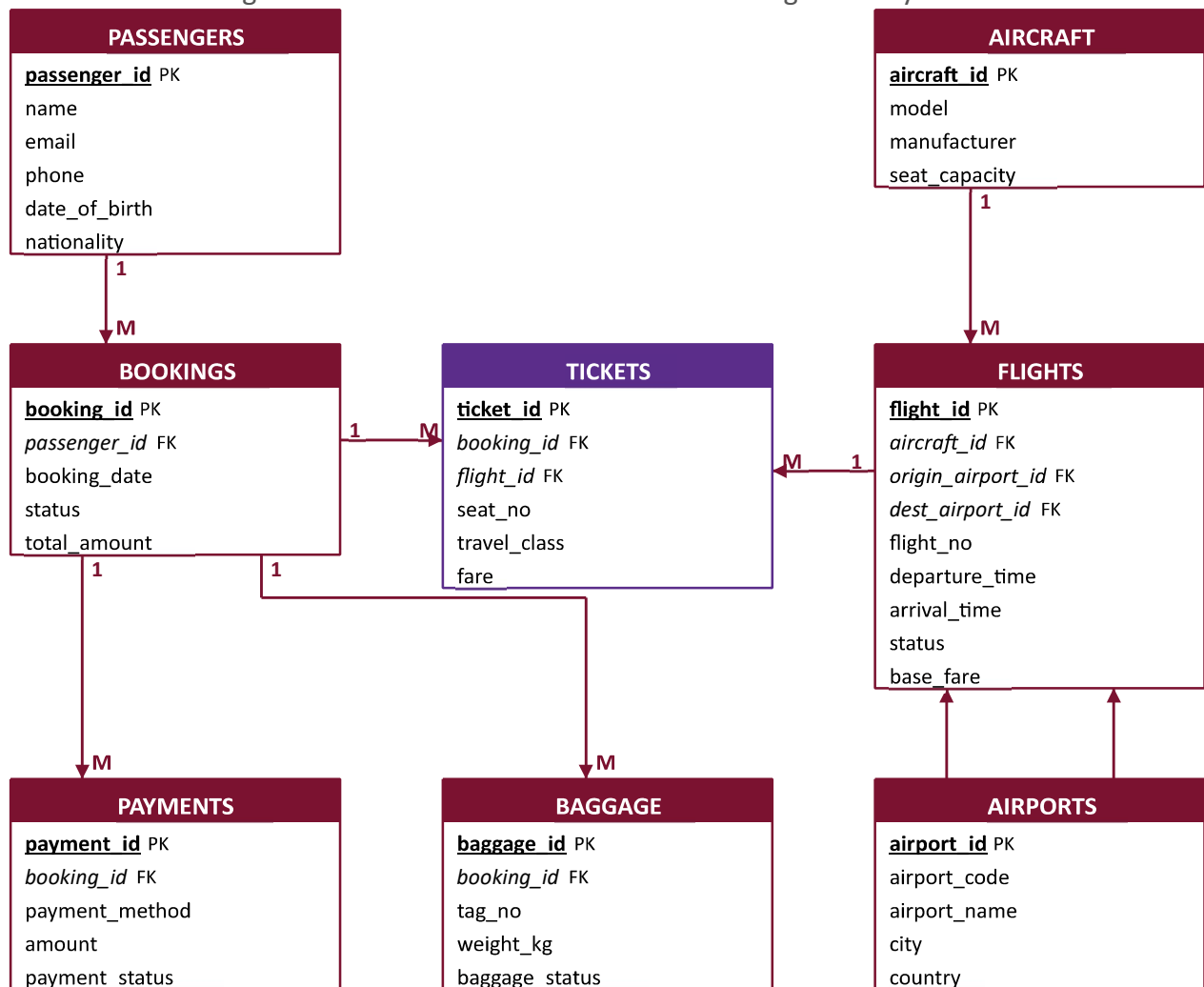


Figure 2 — Relational Schema of the Airlines Management System, showing primary and foreign keys.

◆ PASSENGERS

Column	Type	Key	Description
passenger_id	INT	PK	Unique passenger identifier
name	VARCHAR(100)	—	Passenger full name
email	VARCHAR(120)	—	Login / contact email (unique)
phone	VARCHAR(15)	—	Contact number
date_of_birth	DATE	—	Date of birth
nationality	VARCHAR(50)	—	Nationality

◆ AIRPORTS

Column	Type	Key	Description
airport_id	INT	PK	Unique airport identifier
airport_code	CHAR(3)	—	IATA code (unique)
airport_name	VARCHAR(120)	—	Airport name
city	VARCHAR(80)	—	City
country	VARCHAR(60)	—	Country

◆ AIRCRAFT

Column	Type	Key	Description
aircraft_id	INT	PK	Unique aircraft identifier
model	VARCHAR(60)	—	Aircraft model
manufacturer	VARCHAR(60)	—	Manufacturer
seat_capacity	INT	—	Total seats

◆ FLIGHTS

Column	Type	Key	Description
flight_id	INT	PK	Unique flight identifier
aircraft_id	INT	FK	References Aircraft.aircraft_id
origin_airport_id	INT	FK	References Airports.airport_id (departure)
dest_airport_id	INT	FK	References Airports.airport_id (arrival)
flight_no	VARCHAR(10)	—	Public flight number
departure_time	DATETIME	—	Scheduled departure

arrival_time	DATETIME	—	Scheduled arrival
status	VARCHAR(30)	—	Scheduled / Delayed / Departed / Landed / Cancelled
base_fare	DECIMAL(10,2)	—	Standard fare

◆ BOOKINGS

Column	Type	Key	Description
booking_id	INT	PK	Unique booking identifier
passenger_id	INT	FK	References Passengers.passenger_id
booking_date	DATETIME	—	Date and time of booking
status	VARCHAR(30)	—	Confirmed / Cancelled / Completed
total_amount	DECIMAL(10,2)	—	Total booking value

◆ TICKETS (RESOLVES THE M:N RELATIONSHIP)

Column	Type	Key	Description
ticket_id	INT	PK	Unique ticket identifier
booking_id	INT	FK	References Bookings.booking_id
flight_id	INT	FK	References Flights.flight_id
seat_no	VARCHAR(5)	—	Seat assigned on that flight, e.g. 14A
travel_class	VARCHAR(20)	—	Economy / Premium Economy / Business
fare	DECIMAL(10,2)	—	Fare captured at the time of booking

◆ PAYMENTS

Column	Type	Key	Description
payment_id	INT	PK	Unique payment identifier
booking_id	INT	FK	References Bookings.booking_id
payment_method	VARCHAR(30)	—	UPI / Card / Net Banking
amount	DECIMAL(10,2)	—	Amount paid
payment_status	VARCHAR(30)	—	Pending / Paid / Failed / Refunded

◆ BAGGAGE

Column	Type	Key	Description
baggage_id	INT	PK	Unique baggage identifier
booking_id	INT	FK	References Bookings.booking_id
tag_no	VARCHAR(20)	—	Unique baggage tag number

weight_kg	DECIMAL(5,2)	—	Weight in kilograms
baggage_status	VARCHAR(30)	—	Checked-in / Loaded / Arrived / Collected

05 TASK 5

Relationship Explanation

A plain-language walk-through of each relationship in the ER diagram and why it carries the cardinality it does.

- **Passenger makes Booking (1:M)** — each booking is made by exactly one passenger, but a passenger's travel history can grow without limit, so the "many" side sits on Booking.
- **Booking includes Flight (M:N)** — a booking can cover several flights, such as a return journey or a connection, and any given flight carries many different bookings. The seat_no, travel_class and fare of each leg are properties of the pairing itself, not of the booking or the flight alone, which is why they sit on the relationship rather than on either entity.
- **Booking paid by Payment (1:M)** — one payment usually settles a booking, but the model allows more than one payment row per booking to capture retries, partial payments or refunds, without ever letting one payment settle two bookings.
- **Booking checks in Baggage (1:M)** — a traveller may check in several bags under one booking, while every bag, identified by its tag number, belongs to a single booking.
- **Aircraft operates Flight (1:M)** — an aircraft flies many scheduled flights over its working life, but each individual flight is flown by one aircraft.
- **Airport departs / arrives Flight (1:M, twice)** — an airport can be the origin or destination of many flights, while every flight has exactly one origin and one destination. Airport therefore appears twice in Flight, which is why the schema holds two foreign keys, origin_airport_id and dest_airport_id.

06 SUMMARY

Conclusion

How the design fulfils Assignment 2's requirements.

The Airlines Management System is modelled as seven entities connected by seven relationships. Six relationships are one-to-many; the Booking–Flight relationship is many-to-many and, following standard ER-to-relational conversion rules, is resolved into its own associative table, Tickets. The conceptual ER design and the working relational schema therefore describe the same eight-table database.

Every table carries a single-column surrogate primary key, and every one-to-many relationship is implemented as a foreign key on the "many" side, keeping referential integrity straightforward to enforce and the design easy to extend.

Expected Output

Status

List of entities and attributes	✓ Included
ER Diagram	✓ Included
Relationships and cardinalities	✓ Included
Relational Schema	✓ Included
Primary and Foreign Key identification	✓ Included
Brief relationship explanation	✓ Included